



GREEDY ALGORITHMS

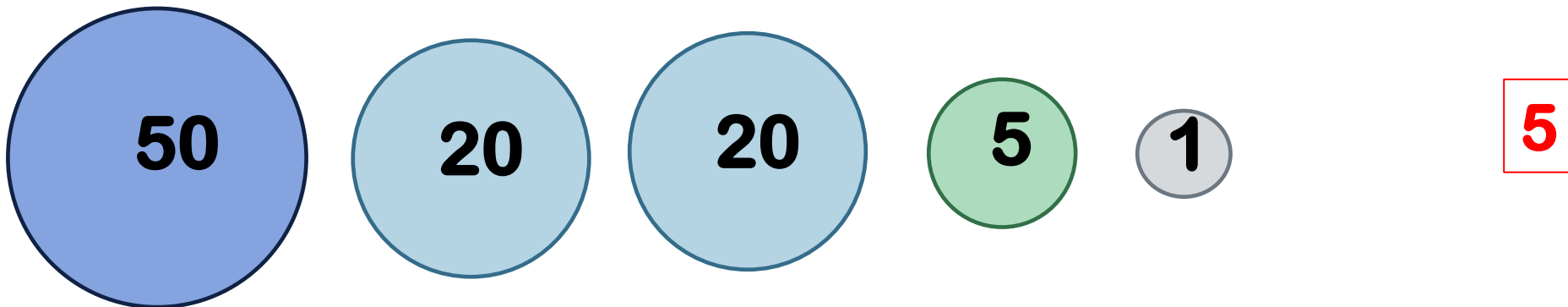
MARZIEH ESKANDARI
NJIT

GREEDY APPROACH

- **Main Idea:** Take the best choice now, hope it leads to the global best.
- 1. **Make a choice** that looks best right now.
- 2. **Check if the problem is solved.**
- 3. **Repeat** until the solution is complete.

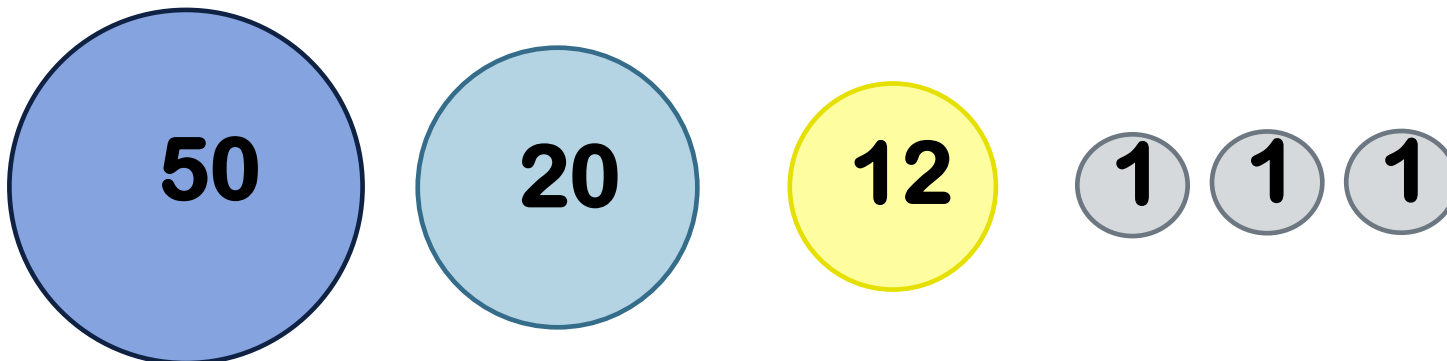
INTRODUCTION

- Given a value of ¢96, and an infinite supply of coins with denominations $\{1, 2, 5, 10, 20, 50, 100\}$, what is the minimum number of coins needed to make change for ¢96?



INTRODUCTION

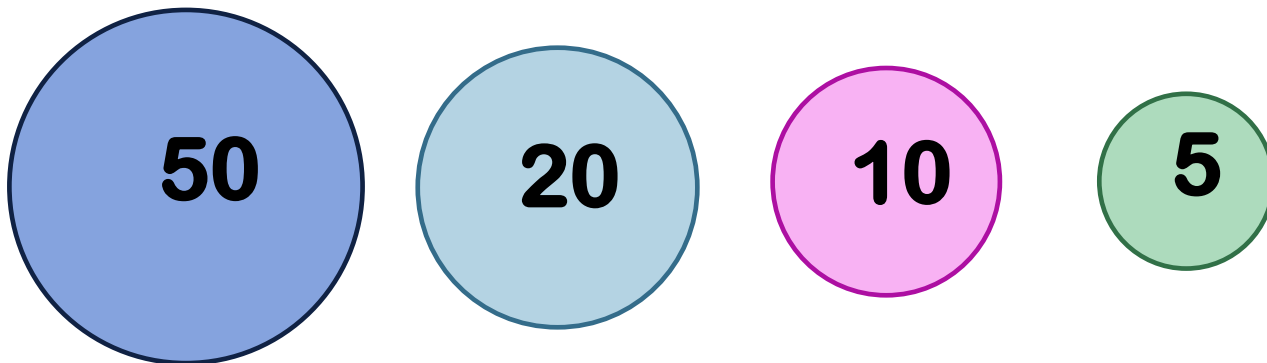
- Given a value of ¢85, and an infinite supply of coins with denominations $\{1, 5, 10, 12, 20, 50, 100\}$, what is the minimum number of coins needed to make change for ¢85?



6

INTRODUCTION

- Given a value of ¢85, and an infinite supply of coins with denominations $\{1, 5, 10, 12, 20, 50, 100\}$, what is the minimum number of coins needed to make change for ¢85?



4

GREEDY APPROACH

Sort C in decreasing order

$S = \emptyset$

while C is not empty:

 if $C.\text{top} \leq V$:

$S.\text{add}(C.\text{top})$

$V = V - C.\text{top}$

 if $V == 0$:

 Print S

 return

$C.\text{delete}()$

KNAPSACK PROBLEM

$$S = \{item_1, item_2, \dots, item_n\}$$

$$w_i = Weight(item_i)$$

$$b_i = benefit(item_i)$$

$$W = Max\ Weight$$

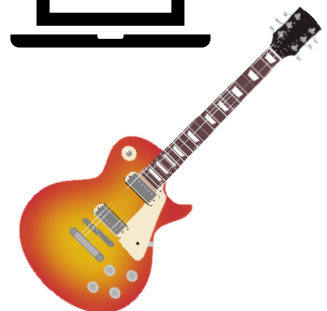
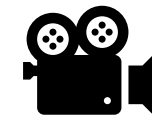
Find a subset of $A \subseteq S$, such that:

The total weight of items in A does not exceed W and the total benefit of items in A is maximized.

Item	Weight	Benefit
Camera	10	320
Laptop	20	400
Watch	8	220
Guitar	12	360



W=20 lbs



0,1-KNAPSACK



W=20 lbs



2



4



1



3

Item	Weight	Benefit
Camera	10	320
Laptop	20	400
Watch	8	220
Guitar	12	360

↓ benefit

Item	Weight	Benefit
Laptop	20	400
Guitar	12	360
Camera	10	320
Watch	8	220

0,1-KNAPSACK



W=-- lbs



Item	Weight	Benefit
Laptop	20	400
Guitar	12	360
Camera	10	320
Watch	8	220

Benefit=400

0,1-KNAPSACK



W=20 lbs



2



4



1



3

Item	Weight	Benefit
Camera	10	320
Laptop	20	400
Watch	8	220
Guitar	12	360



Item	Weight	Benefit
Watch	8	220
Camera	10	320
Guitar	12	360
Laptop	20	400

0,1-KNAPSACK



W=2 lbs

Item	Weight	Benefit
Watch	8	220
Camera	10	320
Guitar	12	360
Laptop	20	400

Benefit=540

0,1-KNAPSACK



W=-- lbs



Item	Weight	Benefit
Camera	10	320
Laptop	20	400
Watch	8	220
Guitar	12	360

Benefit=580

Dynamic Programming?????

FRACTIONAL KNAPSACK

Item	Weight	benefit
1	20	140
2	10	60
3	5	50

W=30 lbs

Item	Benefit
1	140
2	60
3	50

$$140 + 60 = 200$$

Item	Weight
3	5
2	10
1	20

$$50 + 60 + \left(\frac{15}{20}\right) 140 = 215$$

Item	Benefit/Weight
3	10
1	7
2	6

$$50 + 140 + \left(\frac{5}{10}\right) 60 = 220$$

ALGORITHM

1. for each item i : $v_i = \frac{b_i}{w_i}$ (*value index*)

2. $t=0$ //total weight so far

3. $A=\emptyset$ // selected subset

4. While $t < W$ and S is not empty do

 remove an item i with the highest value index

$a_i = \min\{w_i, W - t\}$ //taking more than $W-t$ causes weight overflow

 Add $(item_i, a_i)$ to A

$t = t + a_i$

$S = \{item_1, item_2, \dots, item_n\};$

a_i : represents the amount of item i that is selected to be placed in the knapsack.

TEXT COMPRESSION AND HUFFMAN CODING

- Another application of the greedy method is *text compression*.
- In this problem, we are given a string X defined over some alphabet and we want to efficiently *encode X into a small binary string Y* (using only the characters 0 and 1).

TEXT COMPRESSION AND HUFFMAN CODING

- Standard encoding schemes, such as the ASCII and Unicode systems, **use fixed length binary strings** to encode characters (with 7 bits in the ASCII system and 16 in the Unicode system).
- For example: E: 69=1000101 Z: 90=1011010
- For example, an English document whose length is 100 million characters would require at least 7 megabits to represent in ASCII and 16 megabits to represent in Unicode.

TEXT COMPRESSION AND HUFFMAN CODING

- **Fixed-length code:** Standard encoding schemes, such as the ASCII and Unicode systems, use fixed length binary strings to encode characters

```
100010101000110001011101111110000111  
111101101110110001100110110010010100  
011011001111111000110010110100000101  
10001101011100010100
```

TEXT COMPRESSION AND HUFFMAN CODING

- **Fixed-length code:** Standard encoding schemes, such as the ASCII and Unicode systems, use fixed length binary strings to encode characters

```
100010101000110001011101111110000111  
111101101110110001100110110010010100  
011011001111111000110010110100000101  
10001101011100010100
```

TEXT COMPRESSION AND HUFFMAN CODING

- This is a waste of bits, however, since there are some characters that are hardly ever used and others, like the letter “e”, that **is used so often** that it is shame to be using the same number of bits for it as the seldomly used characters.
- **Variable-length code:** In this scheme, the codes for various characters are allowed to have different lengths. Ideally, we would like the **most-frequently** used characters to use the **fewest number of bits**, and the **least-frequently** used characters to use the **most**.

TEXT COMPRESSION AND HUFFMAN CODING

- We insist that **no code word in our scheme is a prefix of any other code word in our scheme**. Such an encoding is a prefix code.
- The advantage of using such a prefix code is that decoding can be accomplished by using the greedy strategy of processing the bits of Y in order, **repeatedly matching bits to the first code word they represent**.

HUFFMAN CODING

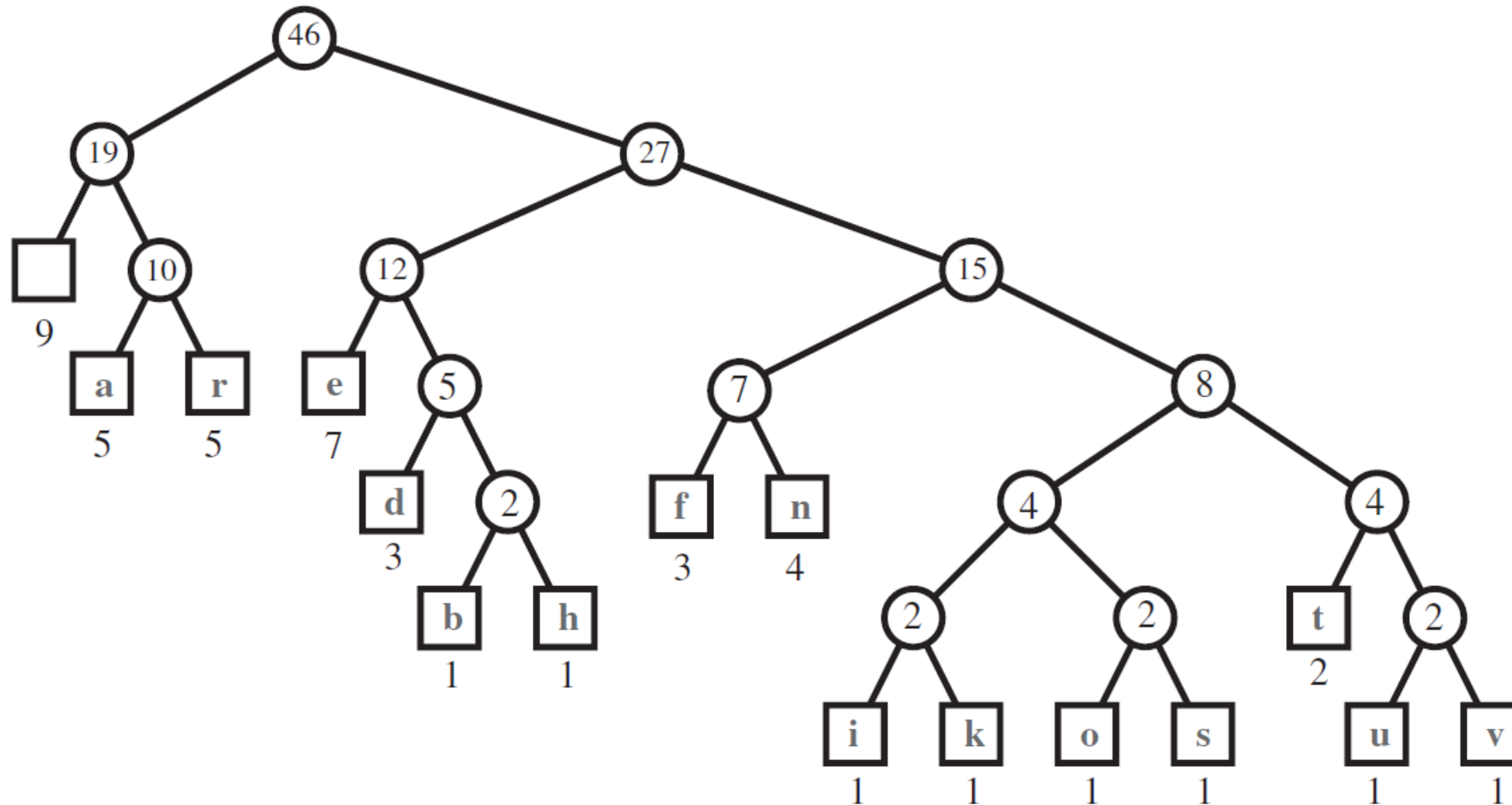
- **Huffman Tree:** This method produces a **variable-length prefix code** for X based on the construction of a **proper binary tree T** that represents the code.
- Each edge in T represents a bit in a code word, with each edge to a left child representing a “0” and each edge to a right child representing a “1.”
- Each external node v is associated with a specific character, and the code word for that character is defined by the sequence of bits associated with the edges in the path from the root of T to v .

EXAMPLE

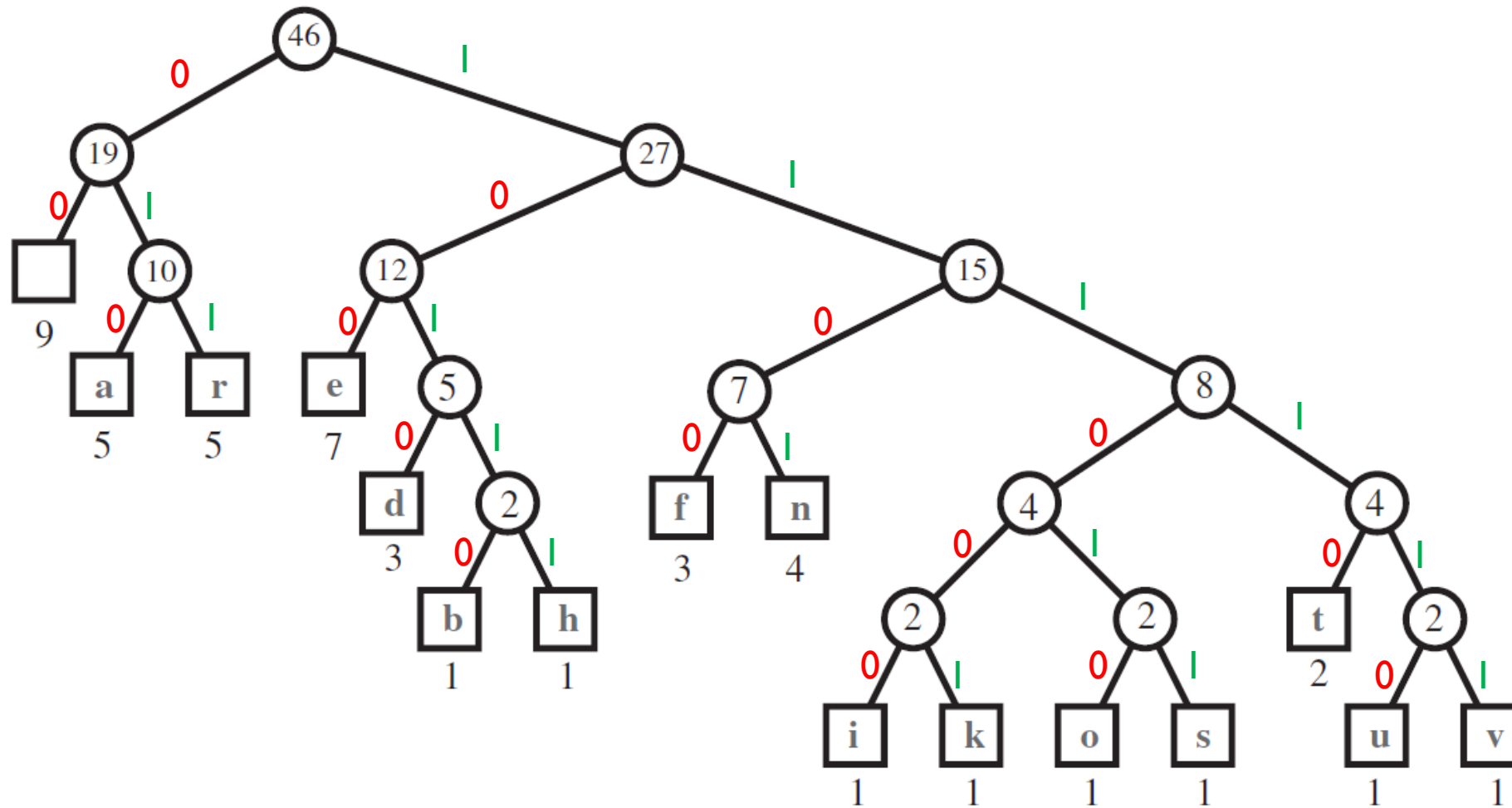
- Encode: text=“a fast runner need never be afraid of the dark”
- $|text|=46$: Unicode: $46 \times 16 = 736$ bits

<i>Character</i>		a	b	d	e	f	h	i	k	n	o	r	s	t	u	v
<i>Frequency</i>	9	5	1	3	7	3	1	1	1	4	1	5	1	2	1	1

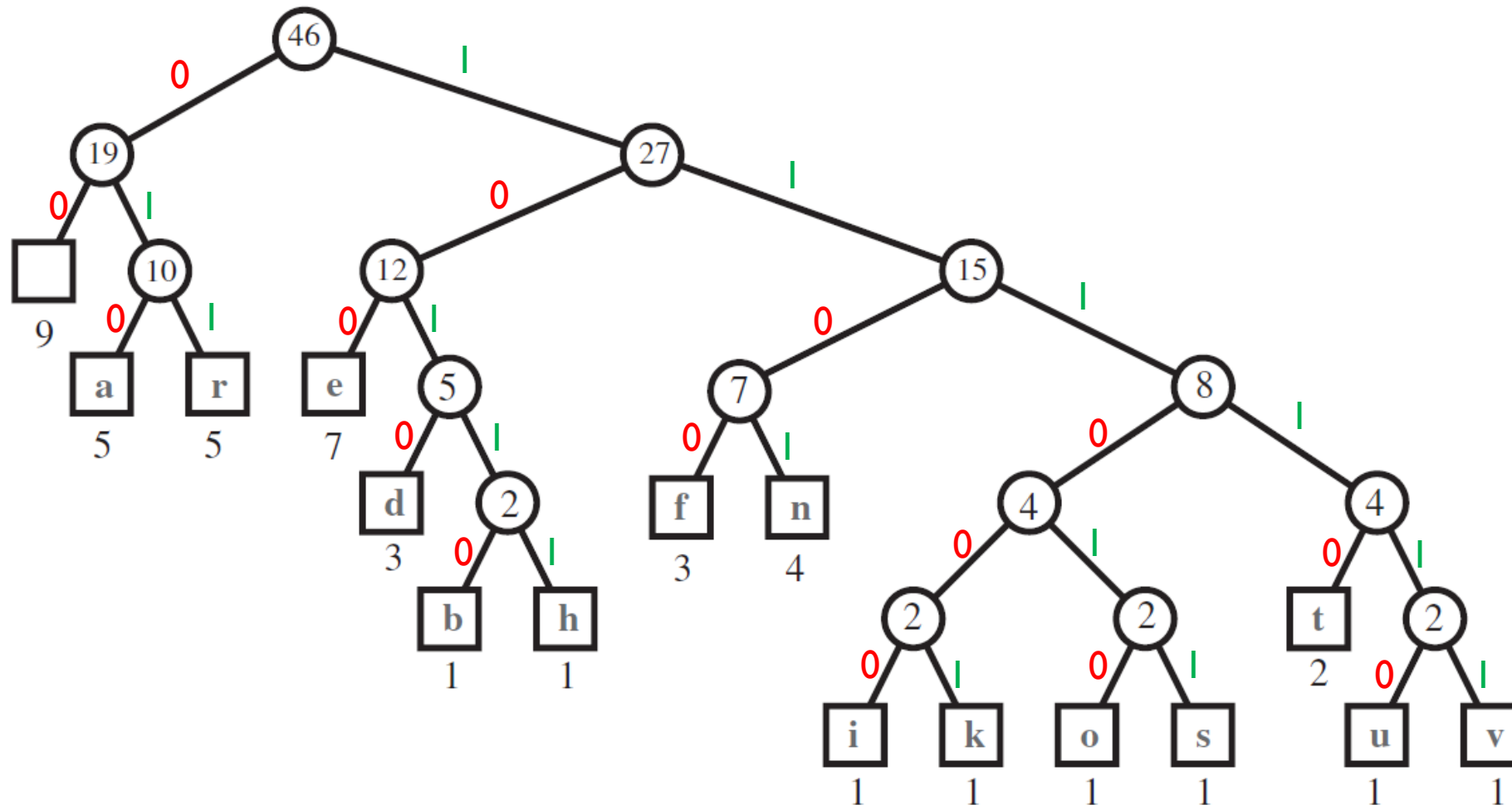
EXAMPLE



EXAMPLE



EXAMPLE



	Code
	00
a	010
r	011
e	100
d	1010
b	10110
h	10111
f	1100
n	1101
i	111000
k	111001
o	111010
s	111011
t	11110
u	111110
v	111111

EXAMPLE

t="a fast runner need never be afraid of the dark"

e=0100011000101110111110000111111011011101100
0110011011001001010001101100111111000110010110
10000010110001101011100010100011101011000011110
101111000010100100111111001

|t|=736 bits

|e|=174 bits

	Code
	00
a	010
r	011
e	100
d	1010
b	10110
h	10111
f	1100
n	1101
i	111000
k	111001
o	111010
s	111011
t	11110
u	111110
v	111111

$26/48$

QUESTIONS!

- How to decode?
- Why Huffman coding is a prefix code?
- Why Huffman coding is a variable-length?
- How to make the tree from table of codewords?

EXAMPLE

character	a	h	o	s	r	m
frequencies	3	4	1	2	1	2



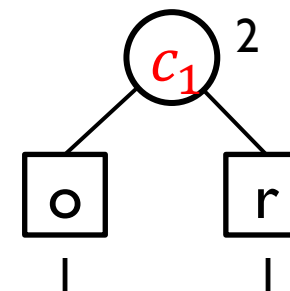
character	o	r	m	s	a	h
frequencies	1	1	2	2	3	4

EXAMPLE

character	o	r	m	s	a	h
frequencies	1	1	2	2	3	4



character	m	s	c_1	a	h
frequencies	2	2	2	3	4

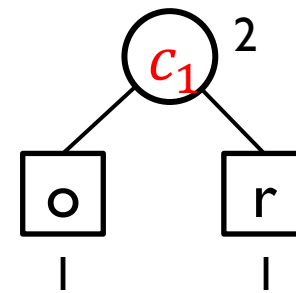
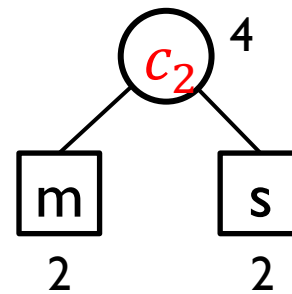


EXAMPLE

character	m	s	c_1	a	h
frequencies	2	2	2	3	4

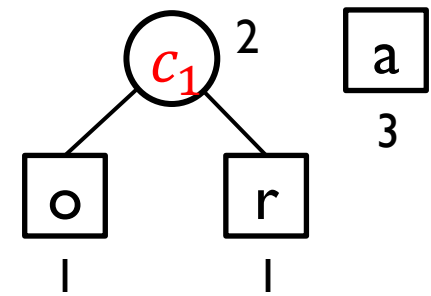
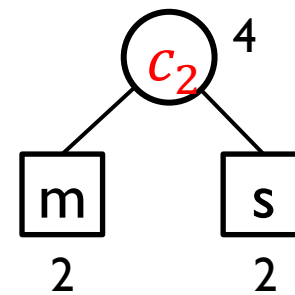


character	c_1	a	h	c_2
frequencies	2	3	4	4



EXAMPLE

character	c_1	a	h	c_2
frequencies	2	3	4	4

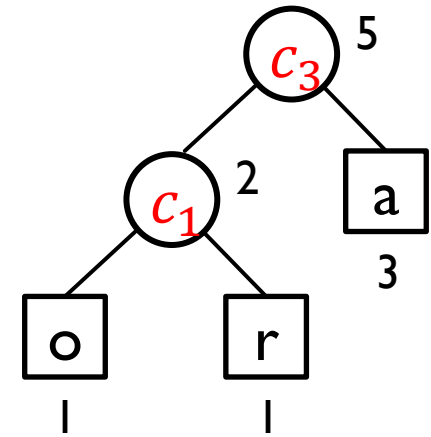
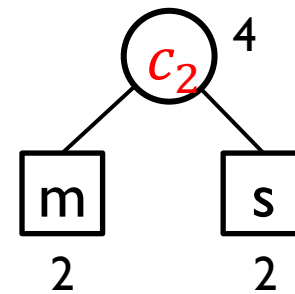


EXAMPLE

character	c_1	a	h	c_2
frequencies	2	3	4	4



character	h	c_2	c_3
frequencies	4	4	5

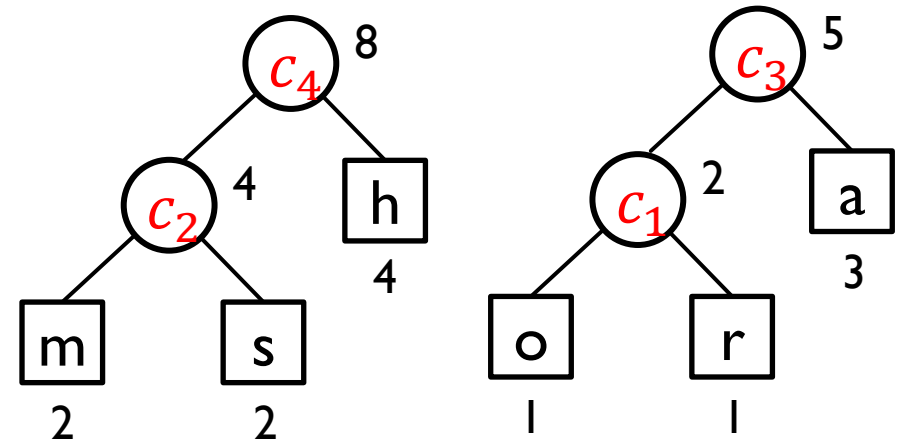


EXAMPLE

character	h	c_2	c_3
frequencies	4	4	5



character	c_3	c_4
frequencies	5	8

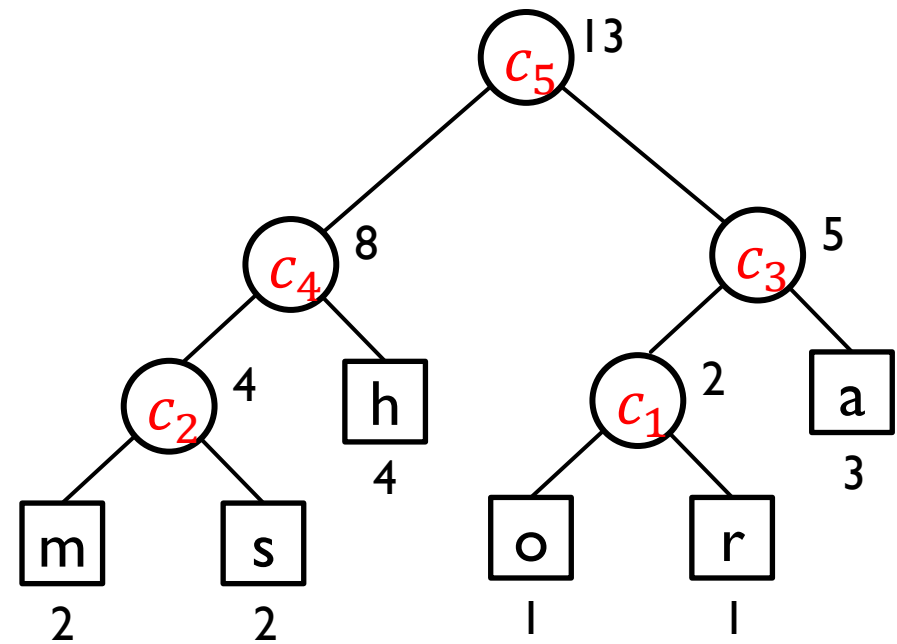


EXAMPLE

character	c_3	c_4
frequencies	5	8



character	c_5
frequencies	13



CONSTRUCT A HUFFMAN TREE

1. Input: A set, C , of d characters, each with a given frequency, $f(c)$
2. Output: a Huffman tree

<i>Character</i>		a	b	d	e	f	h	i	k	n	o	r	s	t	u	v
<i>Frequency</i>	9	5	1	3	7	3	1	1	1	4	1	5	1	2	1	1

CONSTRUCT A HUFFMAN TREE

1. Input: A set, C , of d characters, each with a given frequency, $f(c)$
2. Output: a Huffman tree
3. For each character $c \in C$, create a single-node binary tree T_c storing c .

☐ ☐ a ☐ r ☐ e ☐ d ☐ b ☐ h ☐ f ☐ n ☐ i ☐ k ☐ o ☐ s ☐ t ☐ u ☐ v

CONSTRUCT A HUFFMAN TREE

1. Input: A set, C , of d characters, each with a given frequency, $f(c)$
2. Output: a Huffman tree
3. For each character $c \in C$, create a single-node binary tree T_c storing c .
4. For each character $c \in C$, insert c into a priority queue (MinHeap) Q based on their frequencies, $f(c)$.

u	v	o	s	i	k	b	h	d	e	n	r	a	f	t	
1	1	1	1	1	1	1	1	3	7	4	5	5	3	2	9

CONSTRUCT A HUFFMAN TREE

1. Input: A set, C , of d characters, each with a given frequency, $f(c)$
2. Output: a Huffman tree
3. For each character $c \in C$, create a single-node binary tree T_c storing c .
4. For each character $c \in C$, insert c into a priority queue (MinHeap) Q based on their frequencies, $f(c)$.
5. $c_L = Q.removeMin()$
6. $c_R = Q.removeMin()$
7. Create a new binary tree T with left subtree T_{c_L} and right subtree T_{c_R} .

CONSTRUCT A HUFFMAN TREE

u	v	o	s	i	k	b	h	d	e	n	r	a	f	t	
1	1	1	1	1	1	1	1	3	7	4	5	5	3	2	9

CONSTRUCT A HUFFMAN TREE

u	v	o	s	i	k	b	h	d	e	n	r	a	f	t	
	1	1	1	1	1	1	1	3	7	4	5	5	3	2	9

$$c_L = u$$

CONSTRUCT A HUFFMAN TREE

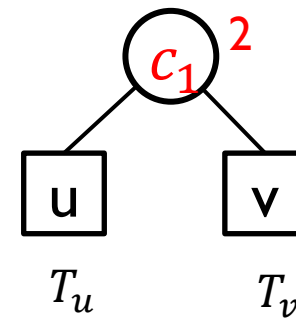
u	v	o	s	i	k	t	b	h		d	e	n	r	a	f
		1	1	1	1	2	1	1	9	3	7	4	5	5	3

$$c_L = u$$

$$c_R = v$$

CONSTRUCT A HUFFMAN TREE

u	v	o	s	i	k	t	b	h		d	e	n	r	a	f
		1	1	1	1	2	1	1	9	3	7	4	5	5	3

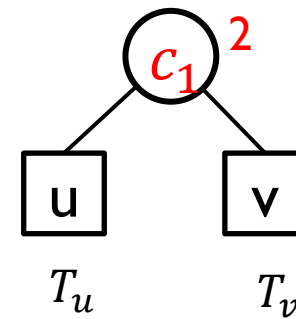


CONSTRUCT A HUFFMAN TREE

1. Input: A set, C , of d characters, each with a given frequency, $f(c)$
2. Output: a Huffman tree
3. For each character $c \in C$, create a single-node binary tree T_c storing c .
4. For each character $c \in C$, insert c into a priority queue (MinHeap) Q based on their frequencies, $f(c)$.
5. $c_L = Q.removeMin()$
6. $c_R = Q.removeMin()$
7. Create a new binary tree T with left subtree T_{c_L} and right subtree T_{c_R} .
8. **Insert a new element into Q with key $f(c_L) + f(c_R)$.**

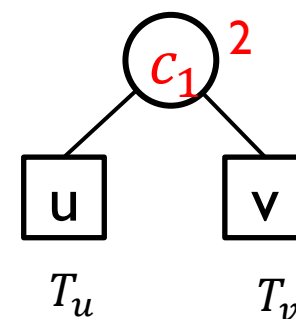
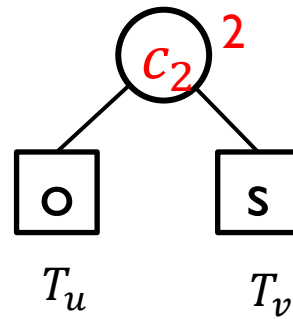
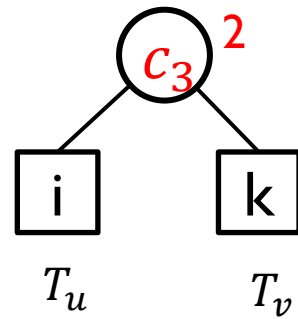
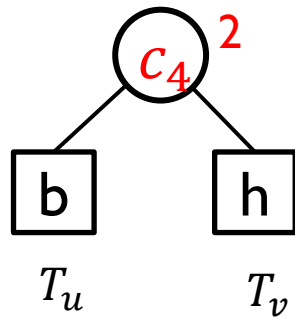
CONSTRUCT A HUFFMAN TREE

u	v	o	s	i	k	t	b	h		d	e	n	r	a	f	c_1
		1	1	1	1	2	1	1	9	3	7	4	5	5	3	2



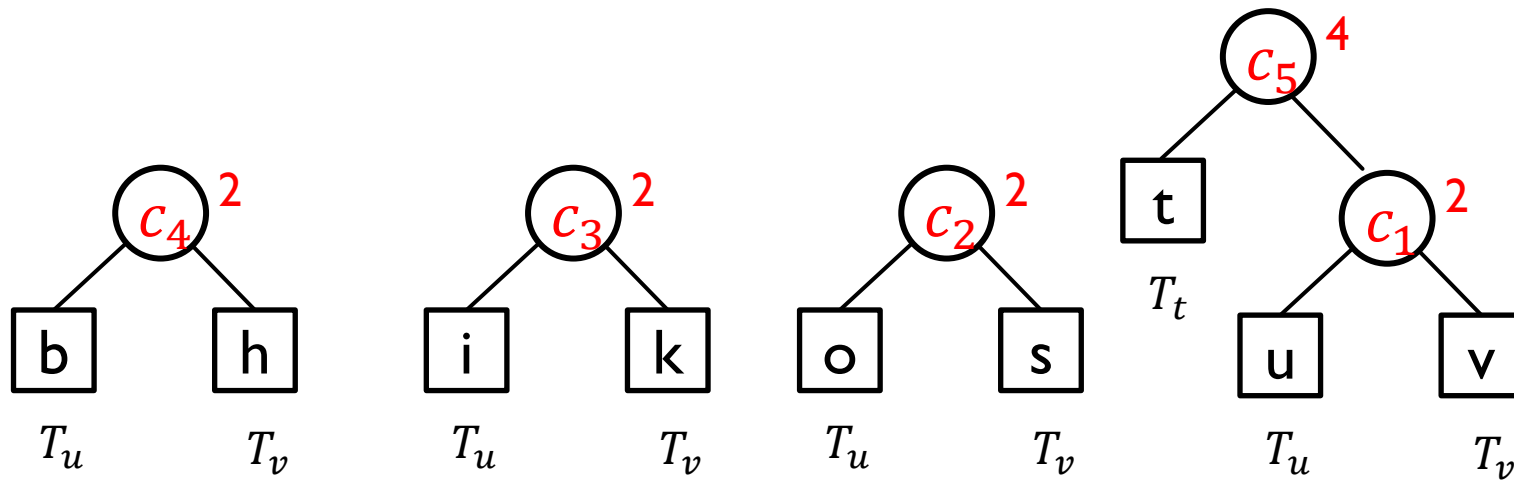
CONSTRUCT A HUFFMAN TREE

u	v	o	s	i	k	b	h	t	c_1	c_3	f	n	c_2	a		d	e	r	c_4
								2	2	2	3	4	2	5	9	3	7	5	2



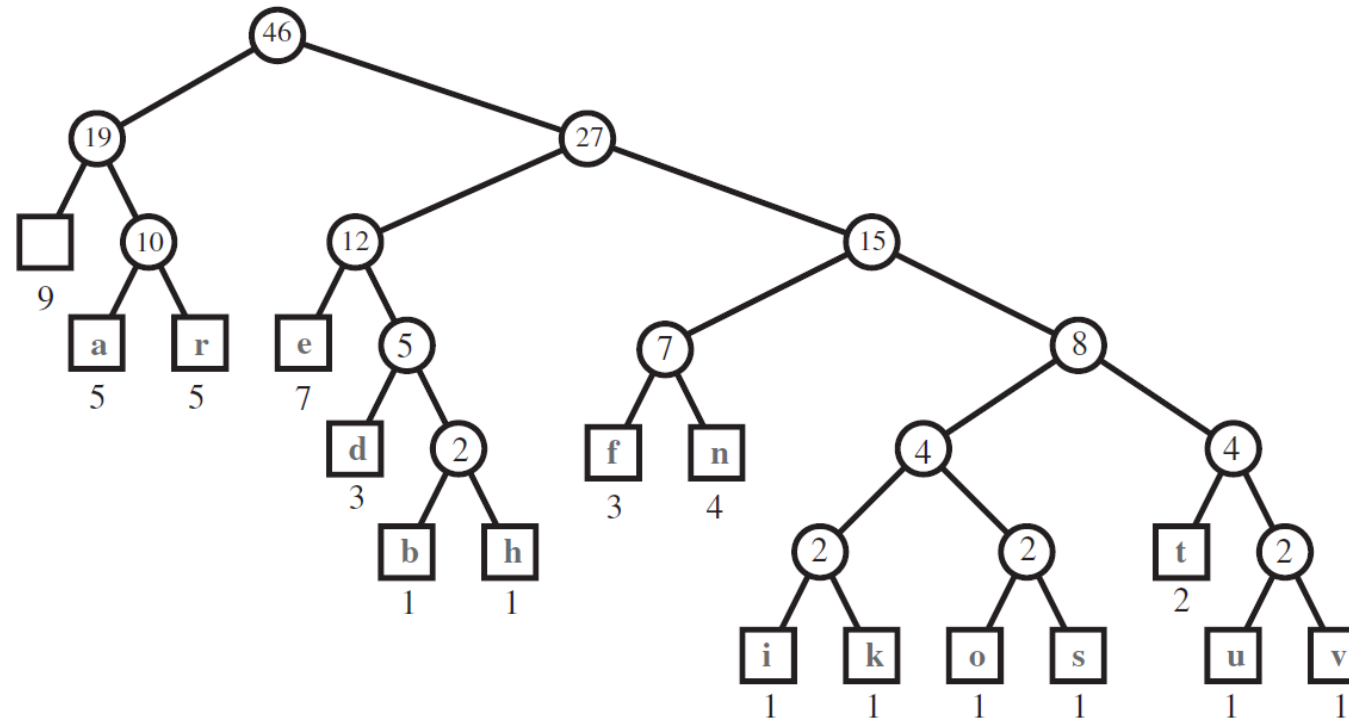
CONSTRUCT A HUFFMAN TREE

u	v	o	s	i	k	b	h	t	c_1	c_3	f	c_2	d	n	c_4	a		r	e	c_5
										2	3	2	3	4	2	5	9	5	7	4



EXAMPLE

Character		a	b	d	e	f	h	i	k	n	o	r	s	t	u	v
Frequency	9	5	1	3	7	3	1	1	1	4	1	5	1	2	1	1



ANY QUESTION??

HAVE A NICE DAY!